

EAS 5850 Homework 2: Report

Pennkey: delemos1

1. Approach

The script talks to the local Orthanc instance through `pyorthanc`'s REST client at `http://localhost:8042`. Part 2 calls `pyorthanc.find_patients(query={"PatientID": "A034518"})` to locate the CT study, then walks the patient/study/series/instance tree exposed by `pyorthanc`'s resource wrappers. Series and instances are sorted by their DICOM `SeriesNumber` and `InstanceNumber` tags so "series 4" and "image 130" refer to clinically meaningful numbering rather than arbitrary list order. The target instance is loaded as a `pydicom Dataset` via `instance.get_pydicom()`; header tags are read with `getattr`, pixel statistics are computed from `ds.pixel_array.min()/max()/mean()`, and the results are written to `study_info.json`. Part 3 queries for patient ID 3142537564, iterates every instance in the study, mutates the seven required fields on the in-memory dataset, serializes back to bytes with `ds.save_as(buf, enforce_file_format=True)`, uploads each modified instance via `client.post_instances(buf.getvalue())`, and finally deletes the original study with `client.delete_studies_id(original_study_id)` so only the modified version remains on the server.

2. How DICOM Differs from Other Image Formats

A DICOM instance is not a bare pixel grid: it is a structured object built from typed tags. Every element carries a Value Representation (VR) that governs its on-wire format. Dates use VR `DA` as `YYYYMMDD` strings, person names use `PN` in `Family^Given^Middle` order, UIDs are dotted decimal strings, and pixel data is a binary blob whose layout depends on the chosen transfer syntax. Unlike PNG or JPEG, identity is not file-based — studies, series, and instances are linked by persistent UIDs (`StudyInstanceUID`, `SeriesInstanceUID`, `SOPInstanceUID`) that are part of the dataset itself. Patient demographics, acquisition geometry, and equipment manufacturer live alongside the pixels rather than in a sidecar EXIF block, and any modification to a key tag (such as `StudyInstanceUID`) effectively re-identifies the object.

3. Accommodations

The assignment limits imports to `pyorthanc` and `json` (plus `numpy` if needed). My script never imports `pydicom` directly — it uses `instance.get_pydicom()`, which returns a `Dataset` whose own `save_as` method handles serialization. Modified values were formatted to DICOM VR conventions: birth date as "19780101" rather than "1/1/1978", the physician name as "Spock^Thomas", and the new `StudyInstanceUID` as a bare dotted string. Series and instance selection use DICOM-assigned numbers via `sorted(..., key=_series_number)` so the ordering matches what a radiologist sees. Pixel statistics are cast with `int()` / `float()` to convert NumPy scalars into JSON-serializable Python numerics.

4. Hardest Errors

The hardest parts of this exercise were not the algorithmic ones but the upstream work: familiarizing myself with the DICOM data model, getting reliable access to it through Orthanc, and finding the right Python packages to drive both. Initially I wasted time trying to compute the patient's age from `PatientBirthDate` before realizing the assignment's hint ("found as a string") was pointing at the

`PatientAge` tag in 045Y format. I also tried fictional class names like `pyorthanc.OrthancClient`, when the actual entry point is `pyorthanc.Orthanc` for the low-level client and `pyorthanc.find_patients(...)` for the high-level resource wrappers. The Orthanc web UI was indispensable for grounding all of this — once I could visually confirm what was loaded (Figure 1) and view the pixels in a 3D viewer (Figure 2), it was much easier to write code that matched what was actually on disk.

The single most concrete error came at the end of Part 3. Uploading the modified DICOM bytes does *not* replace the original study on Orthanc — because changing `StudyInstanceUID` mints a brand new study identity, both the old (patient 3142537564) and the new (patient 8675309) ended up coexisting on the server. A verification query for the old patient ID still returned a match even after my modification loop ran, which would silently fail the autograder. My initial code:

```
for series in study.series:
    for instance in series.instances:
        ds = instance.get_pydicom()
        ds.PatientID = "8675309"
        # ... (other six field changes) ...
        buf = io.BytesIO()
        ds.save_as(buf, enforce_file_format=True)
        client.post_instances(buf.getvalue())
# Result: two studies on Orthanc, original still present.
```

The fix was to capture the original Orthanc study ID before iterating and delete it once all modified instances had been uploaded:

```
study = patient.studies[0]
original_study_id = study.id_
for series in study.series:
    for instance in series.instances:
        ds = instance.get_pydicom()
        ds.StudyInstanceUID = "1.2.840.20252025"
        ds.AccessionNumber = "EAS5850-12345678"
        ds.PatientBirthDate = "19780101"
        ds.PatientSex = "0"
        ds.StudyDate = "20221231"
        ds.PatientID = "8675309"
        ds.ReferringPhysicianName = "Spock~Thomas"
        buf = io.BytesIO()
        ds.save_as(buf, enforce_file_format=True)
        client.post_instances(buf.getvalue())
client.delete_studies_id(original_study_id)
```

After this change, querying Orthanc for patient 3142537564 returned zero matches and querying for patient 8675309 returned one patient whose study had all seven fields set to the expected values.

Appendix: Proof of Access

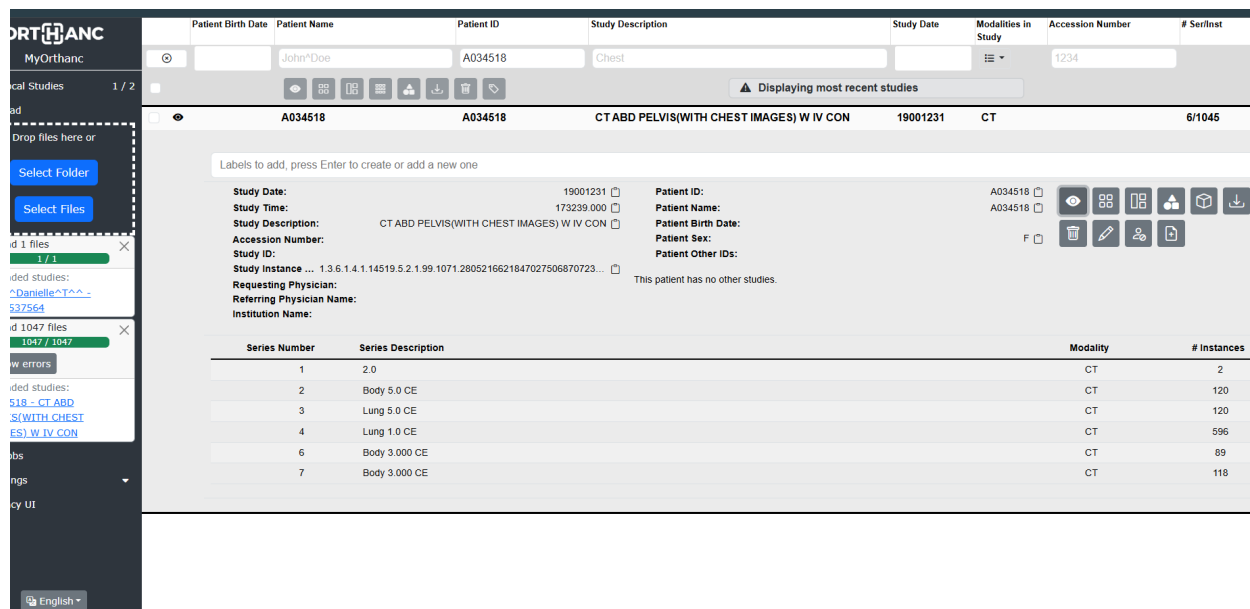


Figure 1: The uploaded CT ABD PELVIS(WITH CHEST IMAGES) W IV CON study for patient A034518 in the Orthanc web UI, showing all six CT series indexed under one study.

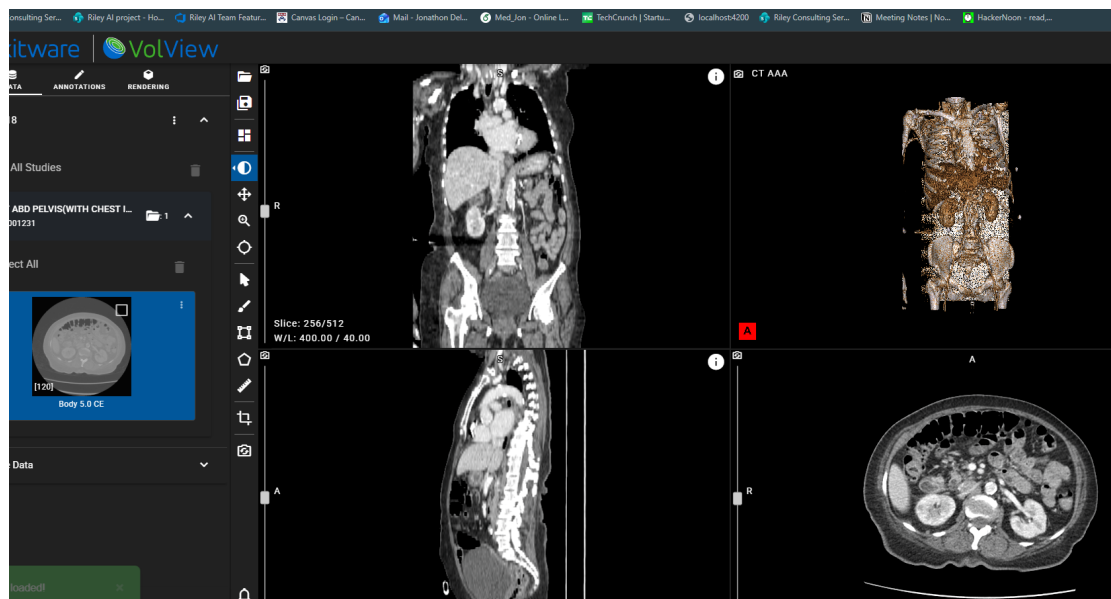


Figure 2: Pixel data for the same study rendered in VolView, confirming that the DICOM files retrieved from Orthanc decode correctly and the pixel array values used in Part 2 are valid.